Optimizing Wireless Sensor Networks Using Dijkstra's Algorithm

Bournemouth University | **COMP7065 - Search & Optimization**
Name – Rushikesh Temghare SRN Number - s5649692

INTRODUCTION

Wireless Sensor Networks (WSNs) consist of small sensor nodes that collect and transmit data. They are widely used in environmental monitoring, smart cities, and disaster management. However, a major challenge in WSNs is ensuring efficient data transmission to base stations while minimising delays.
Optimising routing is crucial for:-
Reducing latency (faster data delivery).
Enhancing energy efficiency (longer sensor lifespan).
Ensuring scalability (handling more nodes effectively).
Without proper optimisation, nodes may take inefficient paths, increasing energy consumption and slowing transmission speeds.

To solve this, Dijkstra's Algorithm is used to determine the shortest and most efficient path from sensor nodes to base stations. It is highly effective because

•It guarantees minimum latency by selecting the shortest available route.

•It works best for static networks, where node positions remain unchanged.
By implementing Dijkstra's Algorithm, WSNs achieve faster data transmission, lower energy usage, and improved network performance.

Problem Definition

Objective:

Find the shortest path from each sensor node to a base station.
Minimise latency by reducing the number of hops in a path.
Maximise transmission rate by selecting links with the highest possible data rate.

Constraints:

Communication Range: Nodes can only communicate if the distance between them is ≤ 3000 meters.
Connectivity Requirement: Every sensor node must find a valid path to a base station.
Latency Accumulation: Each transmission introduces a fixed delay per link, which must be minimised.
Transmission Rate Dependence: The transmission rate between nodes varies based on distance.

Approach:

To achieve these objectives, Dijkstra's Algorithm is implemented to determine the shortest path for each node. This algorithm processes the WSN as a graph, where nodes represent sensors, and edges represent possible communication links based on distance and transmission rate.

Dijkstra's Algorithm

Dijkstra's Algorithm is a widely used method for finding the shortest path in a network. It is particularly effective in static networks, such as our Wireless Sensor Network (WSN), where sensor nodes do not frequently change positions.

Why Use Dijkstra's Algorithm?
Finds the shortest path from sensor nodes to base stations with minimal delay.
Deterministic approach, meaning it always produces the best possible solution.
Efficient for static networks, as it does not require frequent recalculations.

How It Works?
The algorithm treats the WSN as a graph, where:
•Nodes represent sensors
•Edges represent communication links (weighted by latency and transmission rate)

It follows these steps:
Initialise all nodes with a distance of infinity (∞), except for the starting node, which is set to 0. Explore neighbouring nodes and update their distances if a shorter path is found. Use a priority queue to always process the closest node first.
Continue updating distances until the shortest paths to all base stations are determined.
This ensures that each node efficiently transmits data using the fastest and most reliable route.

Methodology

The WSN is represented as a graph, where nodes = sensors and edges = possible transmission links.

Dijkstra's Algorithm is applied to find the shortest path while minimizing latency.

Results are compared with Ant Colony Optimization (ACO) and Genetic Algorithm (GA).

Equations

Latency Calculation:
The total latency LLL is calculated as:

$$\mathbf{L} = \sum_{i=1}^{n} \text{latency per link}$$

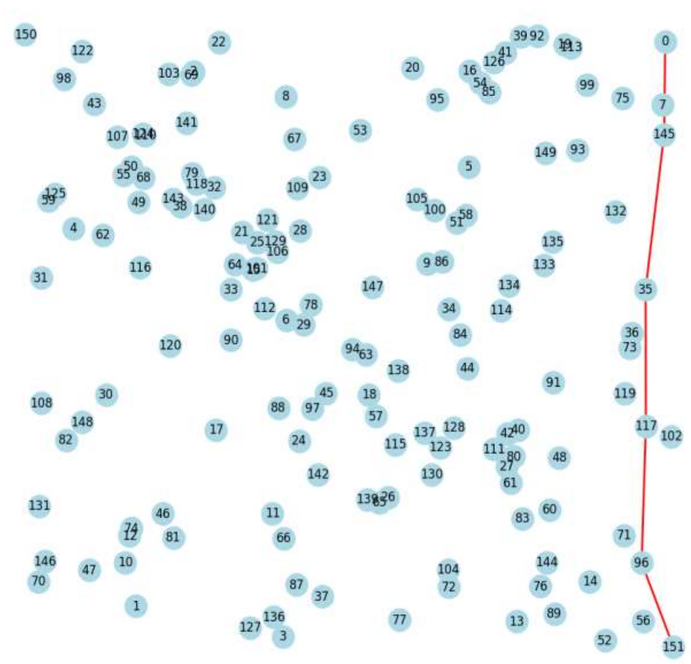
Each hop introduces a delay of **30ms**, which accumulates over multiple hops.

Transmission Rate Formula:
The end-to-end transmission rate is determined by the lowest transmission rate *R* along the path:

$$R = \min(r1, r2, \ldots\ldots\ldots, rn)$$

The weakest link in the path limits the overall transmission speed.

Dijkstra's output Optimal path Visualisation



Results

Performance Comparison:

Algorithm	Avg Latency (ms)	Computation Time (s)
Dijkstra	**85 ms**	**0.02 s**
Ant Colony Optimization ACO	**510 ms**	**1.5 s**
Genetic Algorithm (GA)	**540 ms**	**1.8 s**

Dijkstra's Algorithm is the fastest and most efficient.
ACO & GA take longer but may work better in dynamic networks.
Dijkstra's is best for static WSNs, while ACO/GA may work better in dynamic networks.

Conclusion

Dijkstra's Algorithm efficiently finds the shortest path in a Wireless Sensor Network. Compared to ACO & GA, it has the lowest latency and fastest computation time, making it suitable for real-time WSN applications.

Future Scope

Extending Dijkstra's Algorithm to dynamic WSNs.

Using AI/ML to enhance routing decisions.

Hybrid approaches combining Dijkstra with ACO/GA for better scalability.

References

Akyildiz, I.F., Su, W., Sankarasubramaniam, Y. and Cayirci, E. (2002) 'A survey on sensor networks', IEEE Communications Magazine, 40(8), pp. 102–114.

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009) Introduction to Algorithms. 3rd edn. Cambridge, MA: MIT Press.

Dijkstra, E.W. (1959) 'A note on two problems in connexion with graphs', Numerische Mathematik, 1(1), pp. 269–271.

Tilak, S., Abu-Ghazaleh, N.B. and Heinzelman, W. (2002) 'A taxonomy of wireless micro-sensor network models', ACM SIGMOBILE Mobile Computing and Communications Review, 6(2), pp. 28–36.

Wahyuningsih, D. and Syahreza, E., 2018. Shortest Path Search Futsal Field Location With Dijkstra Algorithm. IJCCS (Indonesian Journal of Computing and Cybernetics Systems), 12 (2), 161.

RESEARCH POSTER PRESENTATION DESIGN © 2022
www.PosterPresentations.com